

# Keychain — Distributed Priority Queue Service

|                   |                                                           |
|-------------------|-----------------------------------------------------------|
| <b>AI tools</b>   | Encouraged — use whatever you normally use                |
| <b>Language</b>   | Your choice — pick what you're most comfortable with      |
| <b>Submission</b> | Git repo (or zip) with a README explaining your decisions |

## 1. Background

Your team needs to build a distributed priority queue service that can be used by multiple backend services to enqueue and process work items. The system must handle concurrent producers and consumers, enforce priority ordering, guarantee at-least-once delivery, and expose metrics for observability and capacity planning.

You are building the Queue Service — the component that owns queues, manages message lifecycle (enqueue, dequeue, acknowledge, retry, dead-letter), and exposes operational metrics.

Choice of datastore is open - in-memory, file, DB you decide. The implementation must be completely thread-safe and testable.

## 2. System Overview

There are three logical roles in scope:

- Queue Service — this is what you build. It owns queues, messages, delivery semantics, and metrics.
- Producers — backend services that enqueue messages with a payload and priority. Treat these as callers of your API.
- Consumers — workers that dequeue messages, process them, and acknowledge completion. Treat these as callers of your API.

You do not need to build real producer or consumer services. Simple test harnesses or integration tests that exercise your Queue Service API are sufficient to demonstrate the system works under concurrency.

## 3. What to Build

Build the Queue Service with the following API surface. The interface style (HTTP, gRPC, library API, etc.) is your choice.

| Operation    | Who calls it  | Purpose                                                                                                |
|--------------|---------------|--------------------------------------------------------------------------------------------------------|
| Create Queue | Admin / setup | Create a named queue with configurable settings (visibility timeout, retry limit)                      |
| Enqueue      | Producers     | Submit a message with payload, priority (HIGH / MEDIUM / LOW), and optional TTL. Returns a message ID. |

|             |            |                                                                                                                                                            |
|-------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dequeue     | Consumers  | Retrieve the highest-priority available message (FIFO within same priority). Marks it <i>"in flight"</i> until acknowledged or visibility timeout expires. |
| Acknowledge | Consumers  | Confirm successful processing of a message by ID. Once acknowledged, the message is permanently removed.                                                   |
| Get Metrics | Monitoring | Expose queue-level metrics (see Section 4)                                                                                                                 |

### 3.1 Functional Requirements

#### Enqueue

- Accept a message with: payload (opaque blob/string), priority (e.g., HIGH, MEDIUM, LOW), and optional TTL or expiration time.
- Return a unique message ID.

#### Dequeue

- Consumers request messages from the queue.
- Should usually receive the highest-priority available message.
- If multiple messages with the same priority exist, use FIFO order.
- Dequeue should mark the message as “in flight” and not visible to other consumers until it is acknowledged (processed successfully), or a visibility timeout expires (consumer crashed or didn’t ack in time), after which the message becomes visible again.

#### Acknowledge

- API to acknowledge successful processing of a message by ID.
- Once acknowledged, the message is permanently removed.

#### Failure & Retry

- If a consumer fails to acknowledge a message within the visibility timeout, the message should be returned to the queue for redelivery.
- Keep track of how many times a message has been redelivered; if it exceeds a configurable threshold, move it to a dead-letter queue.

#### Basic Admin

- Ability to create a named queue.
- Ability to configure per-queue settings (e.g., visibility timeout, retry limit).

## 4. Metrics & Observability

Expose metrics that could be scraped by a monitoring system like Prometheus. You do not need to build the monitoring system itself — only surface the data points and make them queryable.

1. Oldest Message Age — Per queue, the age (in seconds) of the oldest non-expired, non-in-flight message.

2. Message Count — Total messages in the queue (ready to be consumed), broken down by priority (HIGH / MEDIUM / LOW) and optionally by queue name.
3. Messages In Flight — Number of messages currently dequeued but not yet acknowledged, per queue.
4. Throughput — Rate of messages enqueued and acknowledged per second, per queue.
5. Failures / DLQ — Count of messages moved to the dead-letter queue, per queue.

## 5. Non-Functional Requirements

Your design should account for the following, even if the implementation is in-memory:

- The system should work correctly with multiple concurrent producers and consumers (thread safety is mandatory).
- The design should be horizontally scalable — describe in your README how your architecture would extend to multiple instances of the queue service.
- Reasonable latency for enqueue/dequeue (assume  $p95 < 100\text{ms}$  within a region as a design target).
- Durability story — messages should not be silently lost. Describe in your README what guarantees your implementation provides and what a production version would add.
- Node failure handling — document how your design would handle one queue instance going down.

## 6. Deliverables

| # | Deliverable                  | What we are looking for                                                                           |
|---|------------------------------|---------------------------------------------------------------------------------------------------|
| 1 | Working implementation       | Functional, thread-safe queue service — correctness over completeness                             |
| 2 | Data model / internal design | Clean in-memory structures with sensible concurrency controls                                     |
| 3 | Tests                        | Concurrency tests, edge-case coverage, and a clear testing methodology                            |
| 4 | README                       | Your key decisions, trade-offs, scalability story, and what you would do with more time           |
| 5 | Producer / Consumer stubs    | Simple scripts or test harnesses demonstrating the enqueue → dequeue → ack flow under concurrency |

## 7. What We Are Evaluating

- Correctness first — does priority ordering hold? Does at-least-once delivery work? Is the visibility timeout enforced?
- Concurrency & thread safety — can multiple producers and consumers operate simultaneously without data corruption or race conditions?
- Data model clarity — are the internal structures clean, well-reasoned, and easy to extend?

- Engineering judgment — your README decisions matter as much as your code. How you handle trade-offs between simplicity, performance, and correctness is what distinguishes a principal engineer.
- Test quality — do your tests actually cover the hard cases (concurrent access, timeout expiry, DLQ overflow, priority inversion)?
- Metrics design — are the exposed metrics useful, efficient to compute, and well-structured?
- AI fluency — are you using AI tools effectively, not just prompting for boilerplate?

## 8. Practical Notes

- AI tools are fully allowed (and encouraged). We are interested in your judgment and what you chose to build, not whether you typed every line.
- Everything can be built in-memory. No external database, message broker, or infrastructure is required. A complete runnable docker-compose or equivalent is preferred if you have external dependencies, but a locally runnable service is fine.
- Low-level design decisions (data structures, lock granularity, internal threading model, API style) are intentionally left to your discretion. We want to see how you reason about these choices.
- The follow-up session is a live walkthrough of your submission. We will ask you to extend it in a few ways, so make sure you understand your own code.